TrueAvail

Book cash is not usable cash

1. Проблема

N-ая компания работает с большим количеством денежных потоков:

- Карточные платежи
- Выплаты партнерам
- Эквайринг
- Международные транзакции
- Переход между разными валютами

В итогу возникает проблема – рассинхрон между обязательствами компании и фактической доступностью денег, что видно на следующих показателях:

1) SEPA – 1день
   SWIFT – 2-3 дня
   Card clearing - <5дней
2) Выходные и банковские праздники сдвигают дату расчета и создают кассовые разрывы
3) Капитал лежит на разных корреспондентских счетах, при чем где-то в избытке, где-то в недостатке
4) Компания держит избыточные резервы, например 15 млн.$, чтобы избежать задержек
5) Treasury видит проблему только после замеченного риска

2. Решение (функции нашего проекта)

2.1. Forecast & Data Science

2.1.1. Прогноз ликвидности

Прогнозирует будущие балансы по счетам на несколько дней вперед, показывает возможный дефицит денег

2.1.2. ML-ансамбль прогнозирования

Использование нескольких разных моделей (LightGBM Quantile, sklearn Gradient Boosting Quantile, EWMA baseline) для более устойчивого прогноза

2.1.3. Feature Engineering

Модель получает математические признаки (EWMA, лаги, rolling statistics, календарные события, settlement flags)

2.1.4. Фильтрация аномалий

Аномалии фильтруются устойчивой статистикой, а не ручным порогом

2.1.5. Известные события

Объяснение риска от даты (Циклы выплат продавцу, выходные, дни выплат зарплат, задержки при переводе)

2.1.6. Risk Status

Присваивает статус счетам в зависимости от сценария (консервативный реалистичный и агрессивный)

2.1.7. Расчет HHI

Видимость риска зависимости от одного банка (HHI>0.25)

2.2. Optimization & Risk

2.2.1. CCRI-рейтинг банков

Падение рейтинга банка автоматически ужесточает лимиты

2.2.2. MIP-оптимизатор ликвидности

Используя библиотеку python(PuLP), можно найти дешевые переводы между счетами при ограничениях по балансам и экспозиции

2.2.3. Survival Floor Constraints

Запрещает “спасать”, ухудшая кассовый разрыв, концентрацию

2.2.4. Стресс тестинг

Применяет сценарии Bank Run и TARGET2 Outage к балансам и пересчитывает риск.

2.2.5. Expected Shortfall 99%

Считают среднюю потерю в худшем 1% стрессовых исходов

2.2.6. Governance Gate

Проверка рекомендации оптимизатора

2.2.7. Contingency Plan

Когда перевод невозможен, система дает emergency рюlaybook

2.3. Workflow & Business Value

2.3.1. Recommendation approval

Демо показывает human-approved treasury action, а не автоматический перевод денег.

2.3.2. Explainability

Объясняет риск

2.3.3. ROI summary

Считает at-risk liquidity, avoided gap, released reserve, transfer cost, ES99 и снижение ручной работы.

2.3.4. AI Council

Анализ и итоговый вердикт 5 независимых советников

2.3.5. OpenAI + Claude режим

Мультимодельное управление

2.4. UI & Integration

2.4.1. Demo Snapshot

Главный инструмент для фронтенда: собирает в точку весь cockpit state(вся инфа, метрики и тд)

2.4.2. OpenAPI котракт

Фронтенд пишет UI по контракту, а не по условным договоренностям

2.4.3. UI Handoff Artifacts

Генерирует `demo_snapshot.json`, `demo_metadata.json`, `openapi.json` и handoff summary для дизайна и фронтенда.

2.4.4. Backend Verify

Перед запуском запускает проверку всего бакенда

3. Рынок и целевая аудитория

TAM	6 B$ ARR
SAM	1,88 B$ ARR
SOM	25 B$ ARR через 3-5 лет

Идеальный клиент – regulated payment-scale Финтех компании: EMI, PSP, необанки,

Международные кошельки с большим объемом месячных переводов(50-100 млн. $)

4. Ссылки

Source code нашего проекта - https://github.com/kvozianov/liquidity-control-tower1

Финансовая модель+тул на изменение начальных параметров(файл .html), Бизнес модель приложена с сообщением (файл .docx), финансовая модель(файл .xlsx)

https://drive.google.com/drive/folders/113VjRV810K9HUW9qWahgVkp0yxQH-rYE?usp=sharing

5. Решение fintech кейса от команды Eyesight:

Возьянов Кирилл, Подгорный Андрей, Папка Богдан, Джанизбаев Таир